

PROCEEDINGS OF SPIE

[SPIDigitalLibrary.org/conference-proceedings-of-spie](https://www.spiedigitallibrary.org/conference-proceedings-of-spie)

Range pattern matching with layer operations and continuous refinements

I-Lun Tseng, Zhao Chuan Lee, Yongfu Li, Valerio Perez, Vikas Tripathi, et al.

I-Lun Tseng, Zhao Chuan Lee, Yongfu Li, Valerio Perez, Vikas Tripathi, Jonathan Yoong Seang Ong, "Range pattern matching with layer operations and continuous refinements," Proc. SPIE 10588, Design-Process-Technology Co-optimization for Manufacturability XII, 105880J (20 March 2018); doi: 10.1117/12.2297351

SPIE.

Event: SPIE Advanced Lithography, 2018, San Jose, California, United States

Range Pattern Matching with Layer Operations and Continuous Refinements

I-Lun Tseng, Zhao Chuan Lee, Yongfu Li, Valerio Perez, Vikas Tripathi, and Jonathan Yoong
Seang Ong

GLOBALFOUNDRIES Singapore Pte. Ltd.

ABSTRACT

At advanced and mainstream process nodes (e.g., 7nm, 14nm, 22nm, and 55nm process nodes), lithography hotspots can exist in layouts of integrated circuits even if the layouts pass design rule checking (DRC). Existence of lithography hotspots in a layout can cause manufacturability issues, which can result in yield losses of manufactured integrated circuits. In order to detect lithography hotspots existing in physical layouts, pattern matching (PM) algorithms and commercial PM tools have been developed. However, there are still needs to use DRC tools to perform PM operations. In this paper, we propose a PM synthesis methodology, which uses a continuous refinement technique, for the automatic synthesis of a given lithography hotspot pattern into a DRC deck, which consists of layer operation commands, so that an equivalent PM operation can be performed by executing the synthesized deck with the use of a DRC tool. Note that the proposed methodology can deal with not only exact patterns, but also range patterns. Also, lithography hotspot patterns containing multiple layers can be processed. Experimental results show that the proposed methodology can accurately and efficiently detect lithography hotspots in physical layouts.

Keywords: Pattern matching, process hotspot detection, design for manufacturability, design rule checking, PM, DFM, DRC

1. INTRODUCTION

A lithography hotspot pattern is a description for one or a set of geometric patterns, and the geometric patterns are difficult to be produced correctly on silicon wafers during the photolithography stage in a semiconductor manufacturing process. A lithography hotspot is a geometric pattern in a physical layout and the geometric pattern matches a lithography hotspot pattern. Lithography hotspot patterns are usually defined by semiconductor foundries in order to address weaknesses in semiconductor manufacturing processes. Lithography hotspots can act as major factors to cause yield losses of manufactured integrated circuits, especially under aggressive design rules¹. Also, lithography hotspots can exist in a physical layout even if the layout passes design rule checking (DRC). In this article, a lithography hotspot pattern is also called a hotspot pattern.

Since using lithography simulations to detect hotspots in physical layouts is time-consuming, pattern matching (PM) algorithms^{2,3,4} and commercial PM tools have been developed. Although commercial PM tools have been used in the semiconductor industry, there are still needs from specific circuit design companies to use DRC tools to perform PM operations. One of the major reasons for the needs is that most of circuit design companies already have DRC tools. Another reason is that commercial DRC tools are mature, reliable, and efficient. Hence, in order to speed up the process of developing DRC decks (also known as DRC runsets) for the purpose of performing PM operations, it is desirable to automate the process by using software tools. Therefore, in this paper, we propose a PM synthesis methodology for the automatic synthesis of a given lithography hotspot pattern into a DRC deck, so that an equivalent PM operation can be performed by executing the synthesized deck with the use of a DRC tool. Although our PM synthesis methodology incorporates techniques proposed by other researchers, it is different from other DRC-based PM approaches published in the literature^{2,5,6} since our methodology does not require the development of pattern matching algorithms or post-processing steps. Our methodology, however, requires the development of algorithms and software functions to extract geometric and topological characteristics from a given input hotspot pattern, and to generate corresponding layer operation commands based on the characteristics.

The PM synthesis methodology proposed in this paper has many advantages. First of all, the methodology can deal with not only exact patterns, but also range patterns. Secondly, the methodology is able to process hotspot patterns containing multiple layers. Thirdly, the methodology is suitable to be implemented as a software system for synthesizing hotspot patterns into DRC decks. Additionally, with the use of the continuous refinement technique in our methodology, layer operations for extra geometric and topological characteristics of a hotspot pattern can be added into DRC decks conveniently. Finally, by using DRC tools to execute synthesized DRC decks, the efficiency and the parallel computation functionalities of commercial DRC tools can be utilized.

The PM synthesis methodology proposed in this paper consists of four major steps. The first step is to read an input hotspot pattern description file, which is in either GDS or Oasis file format. In the second step, solid polygons contained in the hotspot pattern are partitioned into disjoint rectangles. In the third step, geometric and topological characteristics are extracted from the partitioned rectangles, and corresponding layer operation commands are generated and stored in a DRC deck. Finally, the fourth step of the proposed methodology involves the execution of the generated deck with a DRC tool. By using the proposed methodology, we have successfully synthesized 7nm, 14nm, 22nm, and 55nm lithography hotspot patterns into DRC decks, although the methodology has limitations. Experimental results show that the proposed PM synthesis methodology can accurately and efficiently detect lithography hotspots.

The remainder of this paper is organized as follows. Section 2 defines exact patterns and range patterns. Additionally, the section overviews five layer operations that are frequently used in our PM synthesis methodology. Section 3 presents major techniques that are used in the proposed methodology; these techniques include the continuous refinement technique and the techniques to specify geometric and topological characteristics by using layer operations. Finally, conclusions are drawn in Section 4.

2. PRELIMINARY

A lithography hotspot pattern can be an exact pattern, a range pattern, or a range pattern containing one or more “don’t care” regions⁵. In this section, meanings of exact patterns and range patterns are defined. Note that, in this article, all polygons in hotspot patterns are assumed to be orthogonal polygons⁷.

2.1 Exact patterns

An exact pattern defines one and only one geometric pattern. An exact pattern must have a pattern boundary together with one or more polygons; each polygon belongs to a specific layer. In an exact pattern, furthermore, the pattern boundary and all of the polygons are fixed-coordinate polygons^{8,9}. In other words, each of x and y coordinates for vertices of polygons and for vertices of the pattern boundary has a specific numeric value, and the coordinate value cannot be changed. An example of an exact pattern is illustrated in Figure 1; the pattern contains a pattern boundary (shown in the figure as a rectangular outline whose color is orange) and four solid polygons (shown as blue polygons in the figure). For a given physical layout, the objective of an exact PM operation is to detect all occurrences of layout patterns that match the defined exact pattern.



Figure 1. Example of an exact pattern.

2.2 Range patterns

A range pattern contains more than one geometric pattern since at least one edge of solid polygons in the pattern does not have a fixed location while performing PM operations. A range pattern consists of a pattern boundary, one or more solid polygons with layer information, and one or more marker polygons, which represent tolerance regions in which edges of solid polygons can be moved while performing PM operations. Moreover, these solid polygons and marker polygons belong to fixed-coordinate polygons although locations of edges for solid polygons may be flexible while performing PM operations. An example of a range pattern is illustrated in Figure 2; the pattern contains a pattern boundary (shown in the figure as a rectangular outline whose color is orange), four solid polygons which are colored blue, and four marker polygons which are colored pink. Note that the definition of range patterns described in this article is different from the definition in a paper authored by Yao et al.⁴ since range patterns defined in this article can be stored in GDS and Oasis file formats. Also, in this article, we assume that a range pattern can have one and only one topology¹⁰ for relative positions of polygons in the range pattern. For a given physical layout, the objective of a range PM operation is to detect all occurrences of layout patterns that match the defined range pattern.

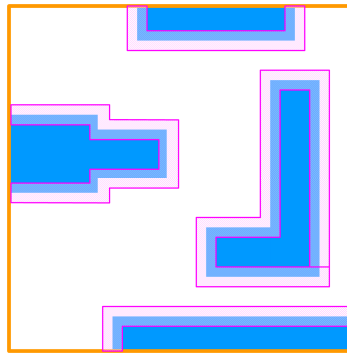


Figure 2. Example of a range pattern.

2.3 Layer operations

In this subsection, we review five layer operations that are frequently used in our PM synthesis methodology, although the methodology also uses other layer operations which are provided by DRC tools.

2.3.1 LENGTH operation

LENGTH is a layer operation which takes a layer and a set of constraints as the input, and returns edges whose lengths meet given constraints. The operation can be expressed as

$$E_O = \text{LENGTH}(L_I, C)$$

where L_I is an input layer, C stands for input constraints, and E_O represents an output layer containing a set of edges. An example of a LENGTH operation is illustrated in Figure 3; input layer L_I contains one (blue) rectangle and output layer E_O contains two (red) edges.



Figure 3. An example of a LENGTH operation.

2.3.2 INTERNAL_REGION operation

INTERNAL_REGION is a layer operation which takes a set of edges and a set of constraints as the input. The operation measures the separation distances between the inward sides of given edges, and returns a layer containing a set of polygons which are formed by the edge pairs with their inward separation distances meeting the constraints. The operation can be expressed as

$$L_O = \text{INTERNAL_REGION}(E_I, C)$$

where E_I is an input layer containing a set of edges, C stands for input constraints, and L_O stands for an output layer. An example of an INTERNAL_REGION operation is illustrated in Figure 4; input layer E_I contains two (red) edges and output layer L_O contains a (green) rectangle which is formed by the two edges.

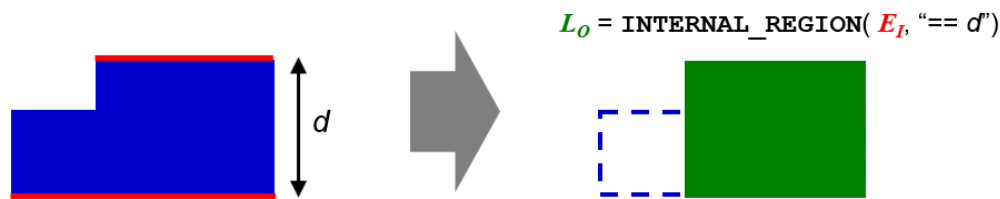


Figure 4. An example of an INTERNAL_REGION operation.

2.3.3 TOUCH operation

TOUCH is a layer operation which takes two layers as the input. The operation returns a layer consisting of polygons from the first input layer and each of these polygons has at least one edge which touches at least one edge of polygons on the second layer. The operation can be expressed as

$$L_O = \text{TOUCH}(L_{I1}, L_{I2})$$

where L_{I1} is the first input layer, L_{I2} is the second input layer, and L_O stands for an output layer. An example of a TOUCH operation is illustrated in Figure 5; input layer L_{I1} consists of three (blue) polygons, input layer L_{I2} contains one (red) rectangle, and output layer L_O contains only one (green) rectangle.

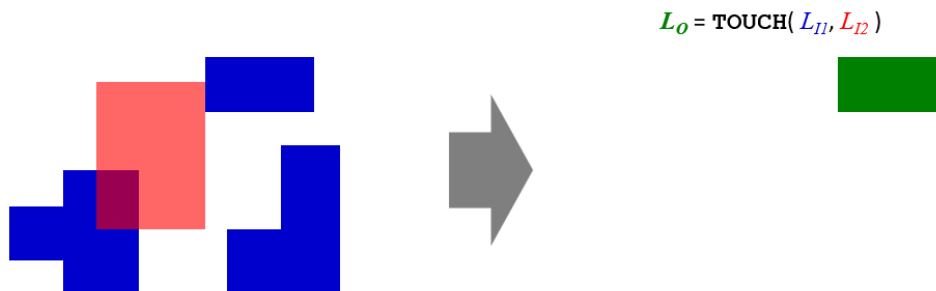


Figure 5. An example of a TOUCH operation.

2.4 INTERACT operation

INTERACT is a layer operation which takes two layers as the input. The operation returns a layer consisting of polygons from the first input layer and each of these polygons shares at least one point with at least one polygon on the second layer. The operation can be expressed as

$$L_O = \text{INTERACT}(L_{I1}, L_{I2})$$

where L_{I1} is the first input layer, L_{I2} is the second input layer, and L_O stands for an output layer. An example of an INTERACT operation is illustrated in Figure 6; input layer L_{I1} contains three (blue) polygons, input layer L_{I2} consists of one (red) rectangle, and output layer L_O contains two (green) polygons.

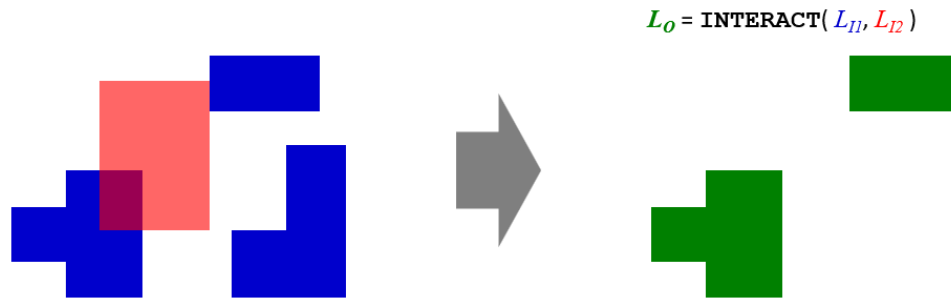


Figure 6. An example of an INTERACT operation.

2.5 EXTERNAL_REGION operation

EXTERNAL_REGION is a layer operation which takes two layers and a set of constraints as the input. The operation measures the separation distances between each edge on the first input layer and each edge on the second layer, and returns polygons which are formed by the edge pairs with their outward separation distances meeting the constraints. The operation can be expressed as

$$L_O = \text{EXTERNAL_REGION}(L_{I1}, L_{I2}, C)$$

where L_{I1} is the first input layer, L_{I2} is the second input layer, C stands for input constraints, and L_O represents an output layer. An example of an EXTERNAL_REGION operation is illustrated in Figure 7; input layer L_{I1} contains one (blue) rectangle, input layer L_{I2} consists of one (red) rectangle, and output layer L_O contains one (green) rectangle.

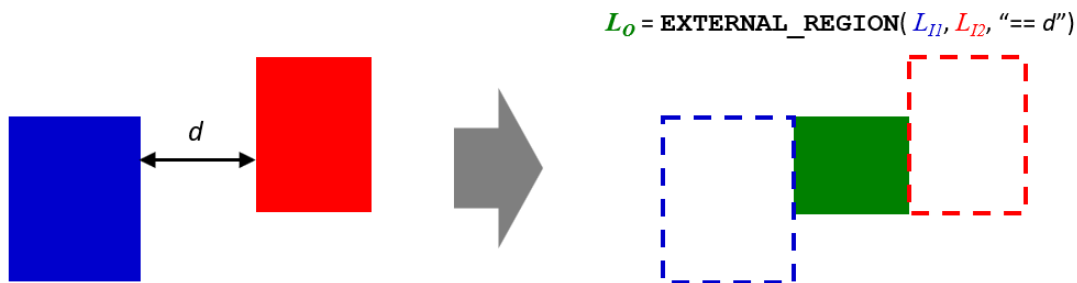


Figure 7. An example of an EXTERNAL_REGION operation.

3. PATTERN MATCHING SYNTHESIS

The PM synthesis methodology proposed in this paper consists of four major steps; the flow of the methodology is illustrated in Figure 8. The first step of the proposed methodology is to read an input hotspot pattern description file, which is in either GDS or Oasis file format. In the second step, solid polygons contained in the hotspot pattern are partitioned into disjoint rectangles. In the third step, geometric and topological characteristics are extracted from the partitioned rectangles, and corresponding layer operation commands are generated and stored in a DRC deck. Note that the geometric and topological characteristics we consider include, but are not limited to, (1) dimensions of solid rectangles, (2) abutment relations between solid rectangles, (3) parallel spacing relations between solid rectangles, (4) corner-to-corner spacing relations between solid rectangles, (5) enclosure relations between solid rectangles, and (6) empty spaces. Additionally, with the use of the continuous refinement technique in our methodology, extra geometric and topological characteristics can be added conveniently. Finally, the fourth step of the proposed methodology involves

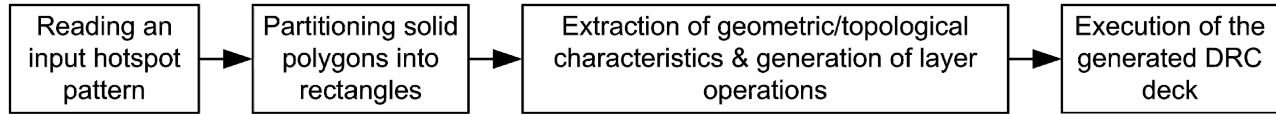


Figure 8. Overall flow of the proposed PM synthesis methodology.

the execution of the generated deck with a DRC tool. This section presents major techniques that are used in the PM synthesis methodology.

3.1 Partitioning orthogonal polygons

In the proposed PM synthesis methodology, solid polygons existing in a hotspot pattern have to be partitioned into disjoint rectangles before layer operation commands are generated. Additionally, each partitioned rectangle is given a unique name. To perform the partitioning process, scanline-based algorithms proposed in the literature^{8,9} can be used. Note that, in our PM synthesis methodology, some orthogonal polygons should be partitioned by using horizontal cuts whereas others should be partitioned by using vertical cuts.

Figures 9(a) and 9(b) illustrate results of partitioning solid polygons existing in hotspot patterns shown in Figures 1 and 2, respectively; these polygons are partitioned by using vertical cuts, which are shown as black dotted lines in the figures. Note that, for range patterns, marker polygons are not required to be partitioned since the information on tolerance ranges of vertex coordinates in partitioned rectangles can be obtained without partitioning marker polygons.

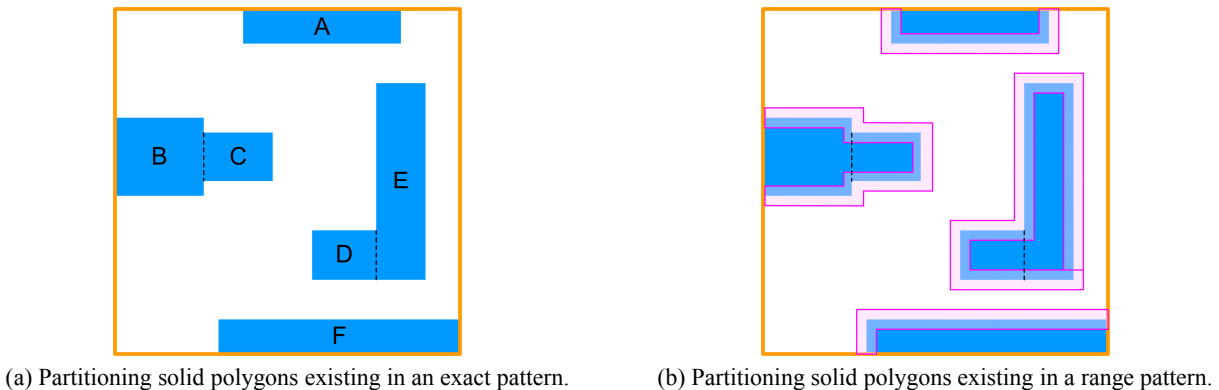


Figure 9. Partitioning orthogonal solid polygons into disjoint rectangles.

3.2 Exact rectangles not touching pattern boundary edges

An exact rectangle is a fixed-coordinate rectangle which is either inside a hotspot pattern or partitioned from an orthogonal solid polygon existing in a hotspot pattern. Additionally, the location of each edge in the rectangle does not have a tolerance range. For instance, in Figure 9(a), rectangles A, B, C, D, E, and F are exact rectangles.

Figure 10(a) shows another example of an exact rectangle which is on layer M1; the rectangle's width is w and its height is h . If we assume that the exact rectangle does not touch the pattern boundary and has no other rectangles touching its upper or lower edge, the following pseudo code of layer operations can be generated in order to detect all or partial polygons that match the exact rectangle.

```

R1_L1 = LENGTH( M1, '>= w' )
R1_RE = INTERNAL_REGION( R1_L1, '== h' )
R1_L2 = LENGTH( R1_RE, '== w' )
R1_Candi = INTERNAL_REGION( R1_L2, '== h' )
  
```

After executing the above pseudo code, layout patterns that match the exact rectangle will be stored in layer $R1_Cand_1$, which is a candidate layer for rectangle $R1$. Polygons stored in a candidate layer may initially contain unwanted polygons. However, those unwanted polygons can be filtered out after other geometric and/or topological characteristics are applied.

For the layout shown in Figure 10(b), the above layer operations can be used to detect the rectangle on the left; the matched rectangle is marked by a brown dashed outline in the figure. In other words, output layer $R1_Cand_1$ will contain the matched rectangle. As another example, after running the above pseudo code on the layout shown in Figure 10(c), output layer $R1_Cand_1$ will be empty since the layout does not match the exact rectangle.

Figure 11 illustrates two abutted exact rectangles on layer $M1$; these two rectangles are partitioned from a polygon which is in a hotspot pattern. Note that one rectangle is above the other one. The following layer operations can be generated for the exact rectangle on the bottom ($R2$) in order to detect matched rectangles in a given physical layout.

```

R2_L1 = LENGTH( M1, '>= h' )
R2_RE = INTERNAL_REGION( R2_L1, '== w' )
R2_L2 = LENGTH( R2_RE, '== h' )
R2_Cand1 = INTERNAL_REGION( R2_L2, '== w' )

```

The output layer named $R2_Cand_1$ is a candidate layer for rectangle $R2$ and will contain all of matched rectangles. Note that we cannot use the techniques presented in this subsection to generate layer operations for an exact rectangle if the rectangle has one of the following properties:

- (1) At least one of its edges touches the pattern boundary.
- (2) At least one of its upper and lower edges touches another rectangle on the same layer, and at least one of its left and right edges touches another rectangle on the same layer.

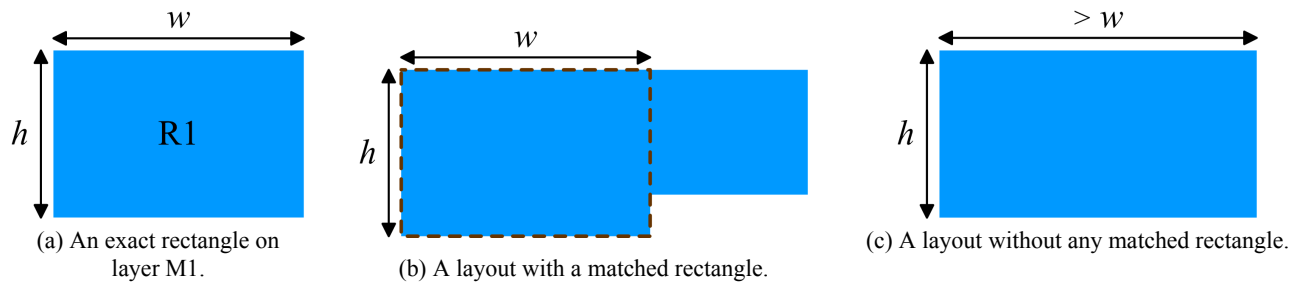


Figure 10. Matching of exact rectangles.

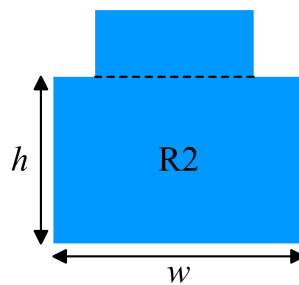


Figure 11. An exact rectangle whose upper edge touches another exact rectangle.

3.3 Range rectangles not touching pattern boundary edges

A range rectangle is a fixed-coordinate rectangle which is either inside a hotspot pattern or partitioned from an orthogonal solid polygon existing in a hotspot pattern. Additionally, the location of at least one edge in the rectangle has

a tolerance range, which is defined by marker polygons in the hotspot pattern. An example of a range rectangle is illustrated in Figure 12(a); the following pseudo code of layer operations can be used for detecting matched rectangles in a physical layout.

```

r = 1
FOR  $h_r = hm$  to  $hp$ :
  R3_L1r = LENGTH( M1, ' $\geq wm$ ' )
  R3_REr = INTERNAL_REGION( R3_L1r, ' $== h_r$ ' )
  R3_L2r = LENGTH( R3_REr, ' $\geq wm \ \&\ \leq wp$ ' )
  R3_Cand1r = INTERNAL_REGION( R3_L2r, ' $== h_r$ ' )
  r = r + 1

```

After executing the above pseudo code, output layer R3_Cand1_r will contain all of matched rectangles. However, the above pseudo code assumes that (1) the range rectangle does not touch the pattern boundary, and (2) the range rectangle's upper and lower edges do not touch any other rectangle which is on the same layer. After running the pseudo code on the layout shown in Figure 12(b), the rectangle whose boundary is marked by brown dashed lines will be detected as a matched rectangle. Note that we cannot use the techniques presented in this subsection to specify a range rectangle if it has one of the following properties:

- (1) At least one of its edges touches the pattern boundary.
- (2) At least one of its upper and lower edges touches another rectangle on the same layer, and at least one of its left and right edges touches another rectangle on the same layer.

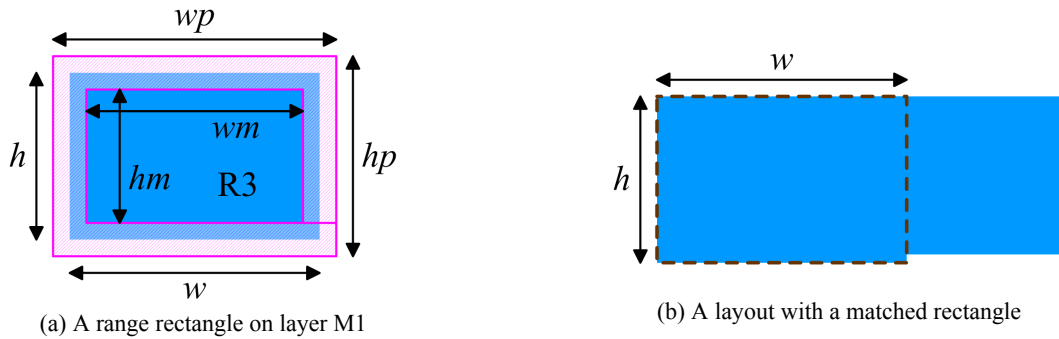


Figure 12. Matching of range rectangles.

3.4 Rectangles touching pattern boundary edges

In this subsection, we use examples to illustrate our techniques for the generation of layer operations when partitioned rectangles touch pattern boundary edges. Since one pattern boundary has four edges, the following three scenarios of a rectangle touching a pattern boundary are considered:

- The rectangle touches one and only one pattern boundary edge
- The rectangle touches two consecutive pattern boundary edges
- The rectangle touches three consecutive pattern boundary edges

In Figure 13(a), rectangle R4 touches one and only one of the pattern boundary edges. In this scenario, rectangle R4 should match partial layout shapes whose widths are w and heights are greater than or equal to h . Also, matched layout shapes should contain two 90-degree corners. Moreover, layout shapes that match rotated and/or mirrored R4 should also be detected. Figure 13(b) shows a physical layout containing three polygons; parts of layout shapes that match R4 are marked by rectangles having brown dashed outlines.

In Figure 13(a), rectangle R5 touches two consecutive pattern boundary edges. In this scenario, rectangle R5 should match partial layout shapes whose widths are greater than or equal to w and whose heights are greater than or equal to h . Also, matched layout shapes should contain a 90-degree corner. Moreover, layout shapes that match rotated and/or mirrored R5 should also be detected. Figure 14 illustrates all of the eight situations for a physical layout when parts of the layout shape match rectangle R5. Note that the physical layout contains a rectangle whose width and height are of the same length; the width and height are greater than w . For each of these eight situations, an output layer is used to store the matched shapes in our PM synthesis methodology.

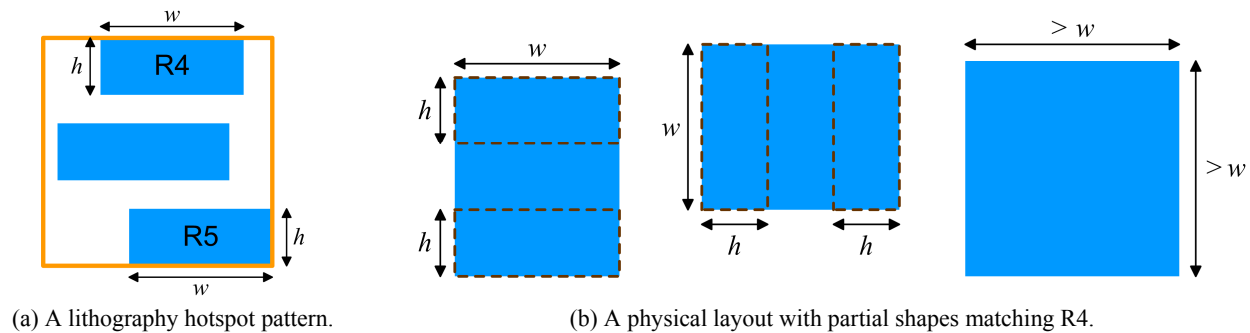


Figure 13. Matching of a rectangle touching one pattern boundary edge.

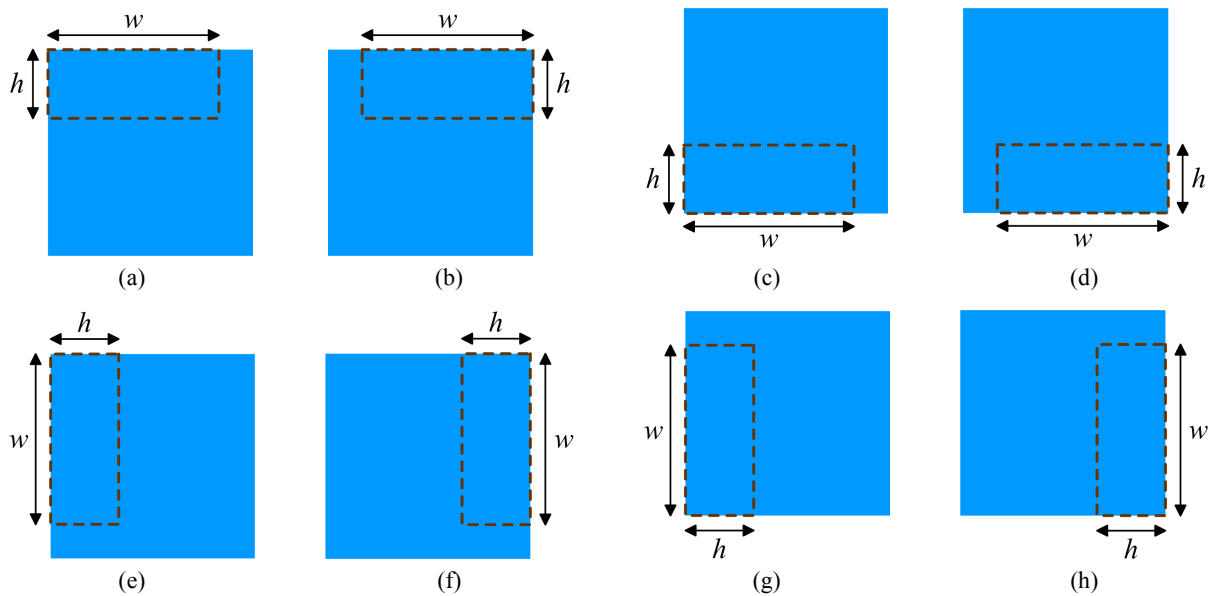


Figure 14. Eight situations for a physical layout with partial shapes matching rectangle R5.

Figure 15(a) shows a hotspot pattern; the pattern contains a rectangle (R6) which touches three consecutive pattern boundary edges. In this scenario, rectangle R6 should match partial layout shapes whose widths are greater than or equal to w and whose heights are greater than or equal to h . Also, matched layout shapes should contain an edge. Moreover, layout shapes that match rotated and/or mirrored R6 should also be detected. Figure 15(b)-(e) illustrates all of the four situations for a physical layout when parts of the layout shape match rectangle R6. Note that the physical layout contains a rectangle whose height is greater than its width; both of its width and height are greater than w . For each of the four situations, an output layer is used to store the matched shapes in our PM synthesis methodology.

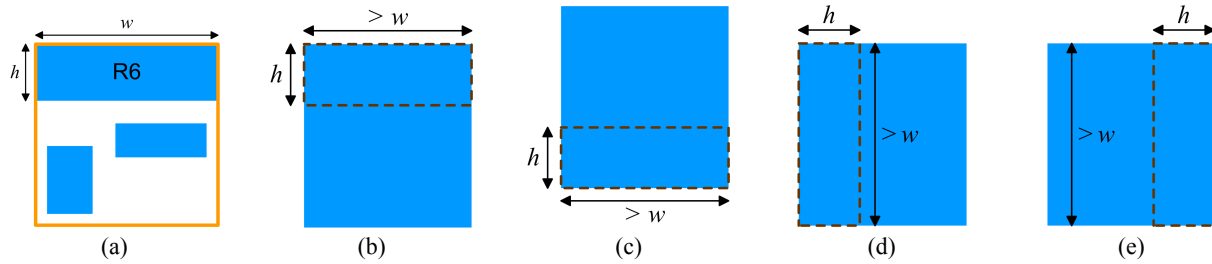


Figure 15. A lithography hotspot pattern containing a rectangle which touches three consecutive pattern boundary edges (a), and four situations for a physical layout with partial shapes matching rectangle R6 (b)-(e)

3.5 The order of processing rectangles

In our PM synthesis methodology, the order of processing partitioned rectangles can be critical. In other words, layer operations for specific rectangles may need to be generated before layer operations for other rectangles can be generated. The example shown in Figure 16 is used to illustrate the importance of the ordering; Figure 16(a) shows an exact pattern, and the polygon in the pattern can be partitioned into rectangles as shown in Figure 16(b). First of all, layer operations for partitioned rectangles C and E can be generated by using techniques described in Subsection 3.2. After that, layout shapes that match the two rectangles can be removed from a given physical layout by using a Boolean-NOT layer operation; the modified layout with regard to the targeted layer can be stored in another layer. Next, layer operations for rectangle B can be generated by using techniques in Subsection 3.2. Furthermore, matched layout shapes can be removed from the layout. Finally, layer operations for rectangles A and D can be generated by using techniques in Subsection 3.4. Note that layer operations for other geometric and topological characteristics should also be generated in order to synthesize a DRC deck for the hotspot pattern in Figure 16(a).

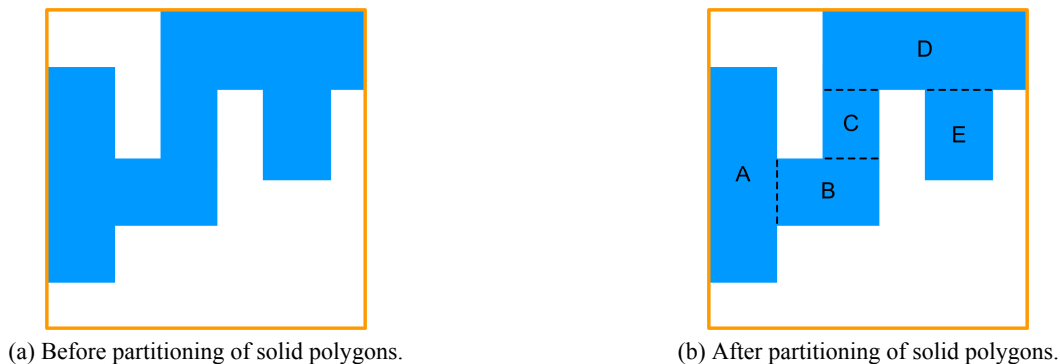


Figure 16. Ordering partitioned rectangles.

3.6 Continuous refinement technique

For a DRC tool, an input DRC deck may not be allowed to have the output layer name of a layer operation command identical to that of another layer operation command. Also, in our PM synthesis methodology, we prefer to continuously refine candidate layers so that polygons on candidate layers can gradually approach our targeted shapes and targeted topologies as layer operations in a DRC deck are executed by a DRC tool. Therefore, in this paper, we propose the use of a technique, which is named the continuous refinement technique. Note that, in Subsections 3.2 and 3.3, part of the continuous refinement technique is used since the following forms of layer names are adopted.

Form 1: *Identifier_Cand_i*
 Form 2: *Identifier_Cand_{i,r}*

The *Identifier* can be the name for a partitioned rectangle or the name for a group of rectangles having specific topologies. In Form 1, subscript *i* denotes a candidate pointer, whose value is an integer. A candidate layer with the

greatest value for the candidate pointer is the current candidate pointer. In Form 2, subscript i denotes a candidate pointer and subscript r represents a variant pointer. Form 2 can be used for layer operations involving range rectangles since a candidate layer can have many variants, as presented in Subsection 3.3. Also, Form 2 can be used for layer operations involving partitioned rectangles which touch pattern boundaries. Our PM synthesis methodology adopts the concept of variants for candidate layers since different polygons may not be stored in a standalone layer.

For Form 1, layer operations using the technique can have the following format:

$$Identifier_Cand_{i+1} = A_layer_operation(\dots, Identifier_Cand_i, \dots)$$

On the other hand, for Form 2, layer operations using the technique can have the following format:

$$Identifier_Cand_{i+1,r} = A_layer_operation(\dots, Identifier_Cand_{i,r}, \dots)$$

In the above two formats of layer operations, i is the current candidate pointer, $i+1$ is the next candidate pointer, and r is a variant pointer.

Figures 17 and 18 are used to illustrate the continuous refinement technique; Figure 17 shows an exact pattern containing partitioned rectangles. Figure 18(a) illustrates a physical layout which contains four polygons; the rectangles marked by brown dashed outlines denote shapes that are included in the initial candidate layer for rectangle R7. Note that the initial candidate layer for rectangle R7 is named R7_Cand₁, as techniques presented in Subsection 3.2 are used. Likewise, the initial candidate layer for rectangle R8 is named R8_Cand₁. As shown in Figure 18(b), the rectangles marked by brown dashed outlines denote polygons that are included in R8_Cand₁.

In Figure 17, since rectangles R7 and R8 are abutted, their abutment relations and their relative locations can be used to filter out polygons that do not match the topological relations. Therefore, by using the continuous refinement technique, the second candidate layer for R7 only contains one rectangle, as shown in Figure 18(c); the second candidate layer is named R7_Cand₂. Similarly, the second candidate layer for R8 is named R8_Cand₂, as shown in Figure 18(d). As more and more geometric and topological characteristics from a given lithography hotspot pattern are extracted and relevant layer operations are generated, a candidate layer with a larger candidate pointer value for a specific partitioned rectangle will contain less unwanted shapes.

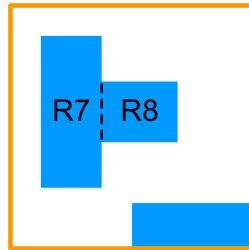


Figure 17. An exact pattern containing partitioned rectangles.

3.7 Abutment relations between rectangles

To specify that two exact rectangles have abutment relations, we can use TOUCH operations together with the continuous refinement technique. For instance, in Figure 9(a), since rectangles D and E are abutted, the following pseudo code of layer operations can be generated.

$$D_Cand_{i+1} = TOUCH(D_Cand_i, E_Cand_j)$$

$$E_Cand_{j+1} = TOUCH(E_Cand_j, D_Cand_i)$$

Checking relative locations between rectangles in D_Cand_{i+1} and rectangles in E_Cand_{j+1}

where i is the current candidate pointer for rectangle D and j is the current candidate pointer for rectangle E.

Note that, to specify the location of rectangle E relative to the location of rectangle D (or vice versa), we use the vector space technique published in the literature². Since the vector space technique requires the computation of (x, y) coordinates for center points of rectangles, the (x, y) coordinates for all possible center points of a range rectangle can

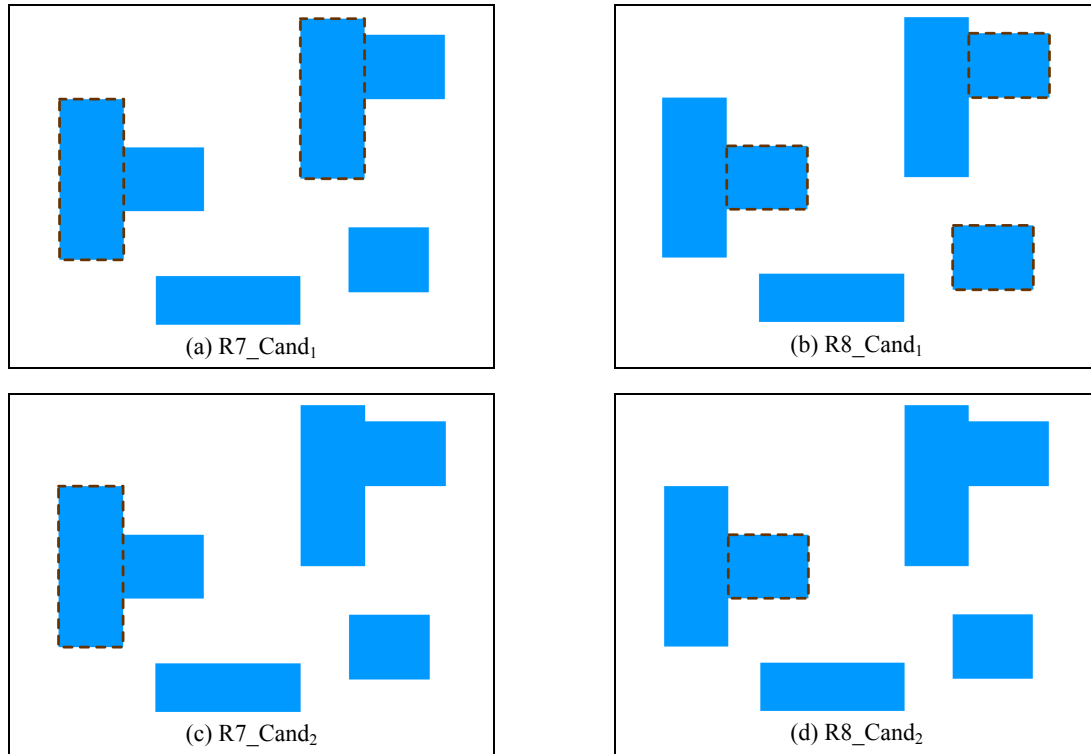


Figure 18. An example illustrating the continuous refinement technique.

have tolerance values. Also, in our PM synthesis methodology, (x, y) coordinates for center points of rectangles touching pattern boundaries are not computed and thus vector spaces for these rectangles are not used. That is because possible locations for all center points of these rectangles can be very wide.

Likewise, if rectangles D and E are range rectangles, the following pseudo code of layer operations can be used to specify their abutment relations:

```

FOREACH  $r$ 
  FOREACH  $s$ 
     $D\_Cand_{i+1,r} = TOUCH(D\_Cand_{i,r}, E\_Cand_{j,s})$ 
     $E\_Cand_{j+1,s} = TOUCH(E\_Cand_{j,s}, D\_Cand_{i,r})$ 
    Checking relative locations between rectangles in  $D\_Cand_{i+1,r}$  and rectangles in  $E\_Cand_{j+1,s}$ 

```

where i is the current candidate pointer for rectangle D, j is the current candidate pointer for rectangle E, r is a variant pointer for rectangle D, and s is a variant pointer for rectangle E. Note that variants of a candidate layer can be merged into one standalone candidate layer by using a Boolean-OR operation if these variants can be merged.

3.8 Parallel spacing relations between rectangles

Two rectangles have parallel spacing relations if the two rectangles do not abut or overlap with each other, and the projection for one edge of a rectangle overlaps with one edge of the other rectangle. For example, rectangles C and E in Figure 9(a) have parallel spacing relations. Thus, the following pseudo code of layer operations can be used to specify the relations.

```

 $M1\_C\_Cand_i = INTERACT(M1, C\_Cand_i)$ 
 $M1\_E\_Cand_j = INTERACT(M1, E\_Cand_j)$ 
 $EXT\_C\_E\_Cand_k = EXTERNAL\_REGION(M1\_C\_Cand_i, M1\_E\_Cand_j, distance\_constraints)$ 

```

$C_Cand_{i+1} = TOUCH(C_Cand_i, EXT_C_E_Cand_k)$
 $E_Cand_{j+1} = TOUCH(E_Cand_j, EXT_C_E_Cand_k)$
 Checking dimensions of $EXT_C_E_Cand_k$
 Checking relative locations between rectangles in C_Cand_{i+1} and rectangles in E_Cand_{j+1}

Note that the continuous refinement technique is used in the above pseudo code. Also, distance constraints can be specified by extracting the real separation distance between rectangles C and E in the lithography hotspot pattern. For example, if the distance between rectangles C and E in the pattern is 100 nm, *distance_constraints* in the above pseudo code should be specified as “== 100 nm”.

Moreover, in the above pseudo code, after the `EXTERNAL_REGION` operation is performed, we can use layer operation commands to check the dimensions of polygons in the output layer of the operation. Also, the vector space technique can be used to check relative locations between rectangles in C_Cand_{i+1} and rectangles in E_Cand_{j+1} .

3.9 Corner-to-corner spacing relations between rectangles

For a corner of a rectangle and a corner of another rectangle, we may specify their corner-to-corner relations. Layer operation commands in DRC tools can be used to generate a rectangle formed by the vertices of two corners whose separation distance is within a specified range. Also, dimensions of the generated rectangles can be checked. In addition, the vector space technique can be used to check relative locations between rectangles containing the two corners if the rectangles do not touch pattern boundary edges.

3.10 Enclosure relations between rectangles

There exist layer operation commands in commercial DRC tools to check whether or not two rectangles have enclosure relations; enclosure distances can also be checked. Additionally, the vector space technique can be used to check relative locations between the two rectangles if the rectangles do not touch pattern boundary edges.

3.11 Consideration of empty spaces

During the process of performing PM operations, empty spaces of a lithography hotspot pattern must also be considered. In other words, a layout pattern matching a hotspot pattern must not contain geometric shapes in empty spaces which correspond to empty spaces in the hotspot pattern. In our PM synthesis methodology, we compute the polygon count in each occurrence of hotspot candidates, and compare the polygon counts with the number of polygons existing in the original hotspot pattern. By using this technique, empty spaces can be checked. Note that hotspot candidates can be formed by merging candidate layers into one layer with the use of Boolean-OR operations.

4. CONCLUSION

In this paper, we have proposed a pattern matching (PM) synthesis methodology for the automatic synthesis of DRC decks from given lithography hotspot patterns. The proposed methodology adopts many techniques, including the continuous refinement technique. Also, we have implemented an initial version of a software system to perform PM synthesis tasks; a commercial DRC tool can be used to execute the synthesized DRC decks in order to perform equivalent PM operations. Note that running the synthesized decks is efficient since many layout operations in the decks generate empty results and the commercial DRC tool has parallel computing capabilities. Our future work includes enhancements of the proposed methodology in order to expand its capabilities.

REFERENCES

- [1] Vito Dai, Jie Yang, Norma Rodriguez, and Luigi Capodieci, “DRC Plus: augmenting standard DRC with pattern matching on 2D geometries,” *Proceedings of SPIE*, Vol. 6521 (2007).

- [2] Jea Woo Park, Robert Todd, and Xiaoyu Song, "Geometric pattern match using edge driven dissected rectangles and vector space," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 35(12), 2046-2055 (2016).
- [3] H. Yao, S. Sinha, and J. Xu, "Efficient range pattern matching algorithm for process-hotspot detection," *IET Circuits, Devices & Systems*, Vol. 2(1), 2-15 (2008).
- [4] H. Yao, S. Sinha, and C. Chiang, "Efficient process-hotspot detection using range pattern matching," *ACM/IEEE ICCAD*, 625-632 (2006).
- [5] Yen-Ting Yu, Iris Hui-Ru Jiang, Yumin Zhang, and Charles Chiang, "DRC-based hotspot detection considering edge tolerance and incomplete specification," *ACM/IEEE ICCAD*, 101-107 (2014).
- [6] Yen-Ting Yu, Ya-Chung Chan, Subarna Sinha, Iris Hui-Ru Jiang, and Charles Chiang, "Accurate process-hotspot detection using critical design rule extraction," *ACM/IEEE DAC*, 1167-1172 (2012).
- [7] I-Lun Tseng and Adam Postula, "An efficient algorithm for partitioning parameterized polygons into rectangles," *ACM GLSVLSI*, 366-371 (2006).
- [8] Yao-Lin Chang and I-Lun Tseng, "A parallel dual-scanline algorithm for partitioning parameterized 45-degree polygons," *ACM Transactions on Design Automation of Electronic Systems*, Vol. 18(4), Article No. 59 (2013).
- [9] I-Lun Tseng and Adam Postula, "Partitioning parameterized 45-degree polygons with constraint programming," *ACM Transactions on Design Automation of Electronic Systems*, Vol. 13(3), Article No. 52 (2008).
- [10] I-Lun Tseng, Adam Postula, and Lech Jozwiak, "Symbolic extraction for estimating analog layout parasitics in layout-aware synthesis," *IEEE MIXDES*, 195-199 (2005).